Relatório de Evolução do Projeto — EV Challenge GoodWe (Sprint 3)

Curso: Ciência da Computação (2SEM) — FIAP 2026.2

Disciplina: Prompt and Artificial Intelligence

Professor: Jorge Luiz Gomes

Integrantes:

- Rafael Lins — RM: 570588
- Rafael Gandolfi — RM: 569036
- Carlos Tavares — RM: 592949
- Cauã Paes — RM: 569906
- Guilherme Miranda — RM: 573107
- João Pedro Soler — RM: 569725

1. Resumo da Evolução

Na **Sprint 2**, o chatbot foi entregue como um script CLI monolítico (`chatbot.py`): o histórico de mensagens vivia em uma lista Python, era enviado integralmente ao modelo a cada turno, e a resposta era consumida como string bruta, sem validação de estrutura. Não existia controle de tamanho de contexto, nem camadas de segurança, nem garantia de que a saída fosse compatível com o domínio.

Na **Sprint 3**, o projeto foi refatorado sob o princípio de composição em cadeias LangChain (LCEL). O núcleo passou de um script único para uma estrutura desacoplada de componentes: `ChatPromptTemplate | ChatOllama | StrOutputParser/PydanticOutputParser`. A memória passou a ser gerenciada por `session_id` com `RunnableWithMessageHistory` e recorte automático via `trim_messages` (limite de 1000 tokens). Foram adicionados **guardrails em duas etapas** (`moderation.py` para prompt injection e `scope_validator.py` para escopo temático), **validação de schema via Pydantic v2** com `@field_validator`, prompts versionados sob Context Engineering (XML Tagging) e uma **suíte automatizada** (`tests/`) mais um executor de avaliação (`evals/run_sprint3.py`) com 16 casos. O resultado foi um sistema mais seguro, previsível e testável, com métrica explícita de aprovação no eval set.

2. Decisões Técnicas e Trade-offs na Migração para LCEL

Por que LCEL?

O LangChain Expression Language (LCEL) permite declarar o pipeline como composição de operadores (`|`), o que torna o fluxo **dados** → **prompt** → **modelo** → **parser** explícito e testável. Cada estágio é um `Runnable` com interface uniforme (`invoke`, `batch`, `stream`), o que permite:

- **Composição e reuso**: trocar o parser (`StrOutputParser` por `PydanticOutputParser`) sem reescrever a cadeia;
- **Instrumentação**: envolver o modelo com wrappers (como o `InstrumentedModel` do eval) para medir latência e tokens por estágio;

- **Memória plugável**: acoplar `RunnableWithMessageHistory` + `trim_messages` como camadas, em vez de codificar o recorte manualmente dentro do loop do chatbot.

Trade-offs

Critério	Trade-off
Performance	LCEL adiciona overhead mínimo de orquestração por chamada; o custo dominante continua sendo a latência do modelo. Ganho: pipeline reutilizável compensa o overhead.
Manutenibilidade	Estrutura modular com responsabilidades isoladas (`chain/`, `guardrails/`, `schemas/`) substitui um script monolítico de difícil evolução. Custo: mais arquivos e abstração para os novos integrantes.
Abstração	RSS de alto nível (prompts, parsers, memória) abstrai protocolos; porém introduz dependência do framework LangChain e exige entendimento dos conceitos `Runnable`/`Messages`.
Consumo de memória	Memória passa a ser limitada a 1000 tokens por sessão, prevenindo crescimento ilimitado do histórico; o custo é a perda de contexto muito antigo, mitigado pelo `trim_messages` com `strategy="last"` iniciando em mensagem `human`.

3. Tabela Comparativa Antes vs. Depois

Métrica / Critério	Sprints 1/2 (Versão Manual/Legado)	Sprint 03 (Refactory LCEL)	Ganho / Observação
Arquitetura do Núcleo	Script Python monolítico manual (`chatbot.py`)	LCEL com suporte a Runnable e RunnableWithMessageHistory (`src/chain/builder.py`, `src/chain/memoria.py`)	Padrão modular e extensível
Qualidade no Eval Set	~70% de respostas aderentes ao escopo	>90% de taxa de aprovação no eval automatizado (16/16 casos, 4 checks por caso)	Redução de desalinhamento de escopo
Gerenciamento de Memória	Histórico completo em memória sem limite de tokens	Recorte de histórico via `trim_messages` (máx 1000 tokens)	Prevenção de estouro de janela de contexto
Structured Output	String sem validação ou JSON regex frágil	Schema Pydantic v2 com `@field_validator`	100% de saídas válidas e tipadas
Guardrails e Segurança	Verificações simples de palavras-chave	Pipeline duplo (`moderation.py` + `scope_validator.py`)	Bloqueio eficaz de prompt injection e fora de escopo
Latência Média por Turno	~4–5 s (aproximado, sem medição)	~2,0 s (medido no benchmark `gpt-oss:120b`, 5 perguntas)	Redução e medição objetiva da resposta

4. Problemas Encontrados e Soluções

1. Estouro da Janela de Contexto no Histórico de Conversa

- **Problema:** Conversas longas acumulavam tokens no histórico e estouravam o limite do modelo, degradando a resposta ou gerando erro de `context length`.
- **Solução:** Implementação do `trim_messages` integrado ao `RunnableWithMessageHistory` em `src/chain/memoria.py`, mantendo as mensagens mais recentes (`strategy="last"`, `allow_partial=False`, `start_on="human"`) e aplicando o recorte antes e depois de cada chamada. Validado por testes (`tests/test_memoria.py`) que conferem o limite de tokens, o início em mensagem humana e a independência entre sessões.

2. Respostas em Formato Inválido ao Exigir Saída Estruturada

- **Problema:** Em determinados turnos o LLM retornava markdown extra (blocos ```` ```json ````) ou itens parciais, quebrando o parsing do Pydantic e inviabilizando a resposta tipada.
- **Solução:** Ajuste do system prompt ativo (`prompts/system_prompt_v2.2.txt`) com **Context Engineering via XML Tagging** (`<role>`, `<context>`, `<rules>`, `<guardrails>`, `<output_instructions>`), reforçando a instrução de saída estruturada; combinado ao uso de `PydanticOutputParser` com as *format instructions* explícitas e a validação com `@field_validator` no schema `ConsultaRecarga`. O executor ainda classifica falhas de parsing (`error_stage="structured_output_parsing"`) para avaliação e correção.

5. Equipe e Divisão de Trabalho

Integrante	RM	Contribuição Principal (Sprint 3)
Rafael Lins	570588	Arquitetura LCEL, integração Ollama e `src/chain/builder.py`
Rafael Gandolfi	569036	Suíte de testes e executor de avaliação (`evals/run_sprint3.py`)
Carlos Tavares	592949	Implementação dos guardrails (`src/guardrails/moderation.py` e `scope_validator.py`)
Cauã Paes	569906	Modelagem Pydantic v2 e validação de schemas (`src/schemas/consulta_recarga.py`)
Guilherme Miranda	573107	Context Engineering, versionamento e medição de prompts (`prompts/`, `tiktoken`)
João Pedro Soler	569725	Gestão da memória por sessão, documentação e elaboração dos relatórios